CPSC 483: Introduction to Machine Learning

Instructor: Anli Ji

Fall 2026

Cal State Fullerton

Chapter 2

Data Preprocessing

- Exploratory data analysis
- Test-train split
- Sampling strategies
- Feature transforms
- Feature scaling
- Handling missing data
- Data processing pipelines

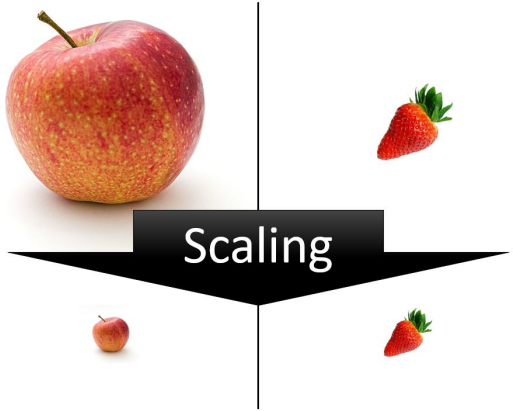
Feature Transformation - *Scaling*

Most ML algorithms don't perform well when input numerical attributes have very different scales

- will be biased toward ignoring the smaller unit features and focus more on the larger unit features

Example

- Age = [18, 80]
- Annual Income = [$20,000, $200,000]



Feature scaling is the process of transforming numerical features to a common scale without distorting differences in the range of values

Feature Transformation - *Scaling*

Min-Max Scaling (i.e., Normalization)

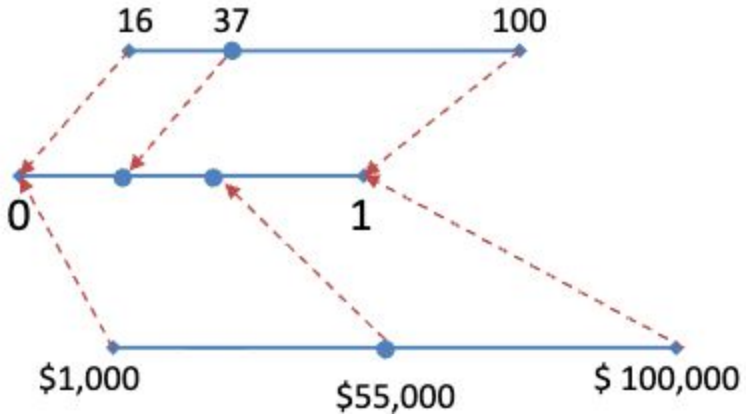
- Convert each feature independently into one comparable range (e.g., 0… 1)
- Percentile:
  - Proportion of the Min to Max range

Mathematically speaking: $$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Example: $$\frac{37 - min}{max - min} = \frac{37 - 16}{100 - 16} = 0.25$$

```
from sklearn.preprocessing import MinMaxScaler

min_max_scaler = MinMaxScaler(feature_range=(0, 1))
feature_scaled = min_max_scaler.fit_transform(original_feature)
```

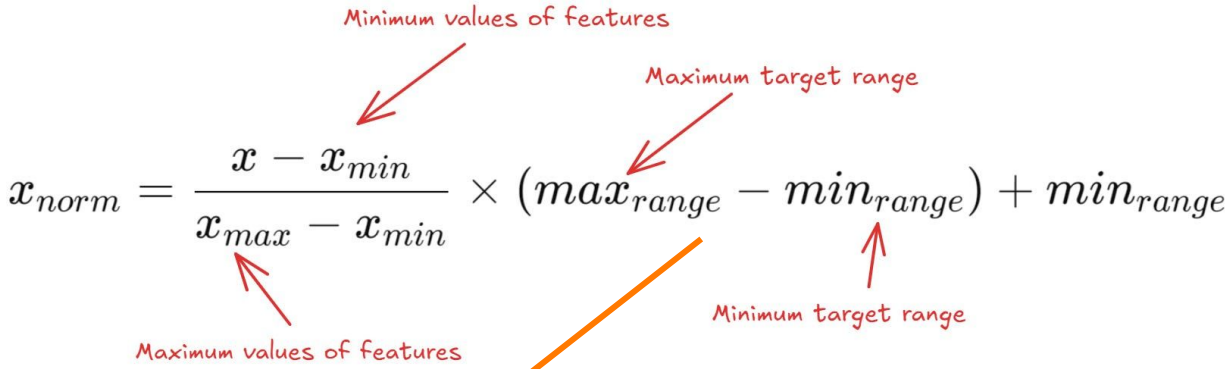


Feature Transformation - *Scaling*

Min-Max Scaling (i.e., Normalization)

- What if different scale?
  - Maybe [-1, 1]?

Mathematically speaking:




```
from sklearn.preprocessing import MinMaxScaler

min_max_scaler = MinMaxScaler(feature_range=(-1,1))
feature_scaled = min_max_scaler.fit_transform(original_feature)
```

Compresses normal values
But highly sensitive to *outliers*

Practice!!!

- What would be the output of applying Min-Max scaling to this dataset?
  - *Range of [0, 1]*
  - *Range of [-1, 1]*

Age	Income
16	10,000
28	2,000
30	6,000
23	5,000
60	25,000

- Do this by hand
- Do this with code
  - First create the *DataFrame*

Feature Transformation - *Scaling*

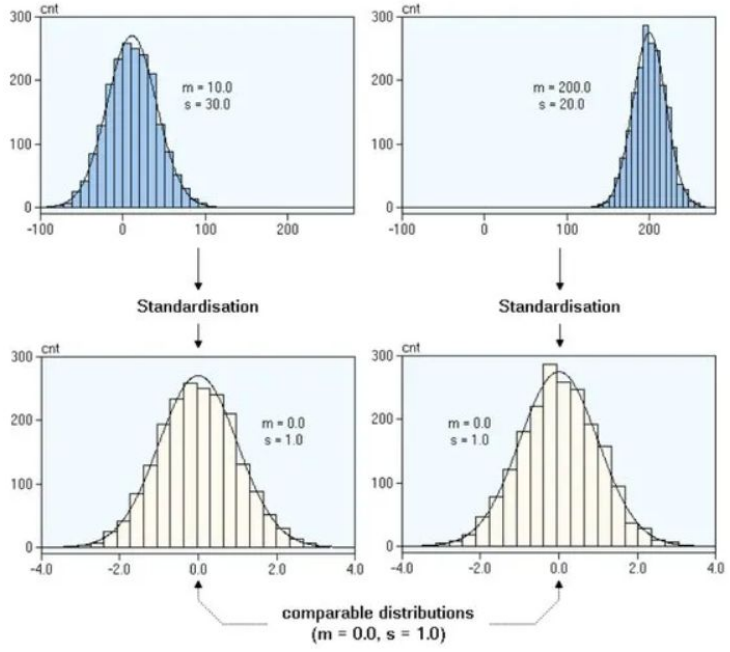
Standardization (i.e., Z-Score Scaling)

- Transforming data to have a mean of 0 and a standard deviation of 1
  - subtract the mean value
  - then divide by the standard deviation
- But range is not limited (can be very large positive or negative values)

Mathematically speaking: $$z = \frac{x_i - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler

std_scaler = StandardScaler()
features_scaled = std_scaler.fit_transform(original_features)
```



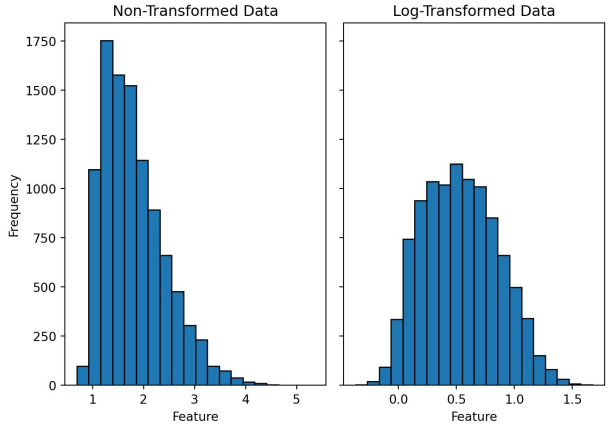
Feature Transformation - *Custom Transformers*

Other: Apply a numerical function to a feature (*Fancier implementation*)

- Combine features to get a new feature that is more related to the target variable
  - i.e., log(), sqrt()

```
from sklearn.preprocessing import FunctionTransformer

log_transformer = FunctionTransformer(np.log, inverse_func=np.exp)
log_pop = log_transformer.transform(housing[["population"]])
```



Example:

Ratio of two variables

- housing["rooms_per_house"] = housing["total_rooms"] / housing["households"]
- housing["bedrooms_ratio"] = housing["total_bedrooms"] / housing["total_rooms"]

Numerical function

- housing["log_total_rooms"] = np.log(housing["total_rooms"])

Create a Test Set - *Sampling*

Split data into *train-test* subsets

- Do exploratory data analysis ONLY on the **train** subset
  - Avoid *data snooping bias*

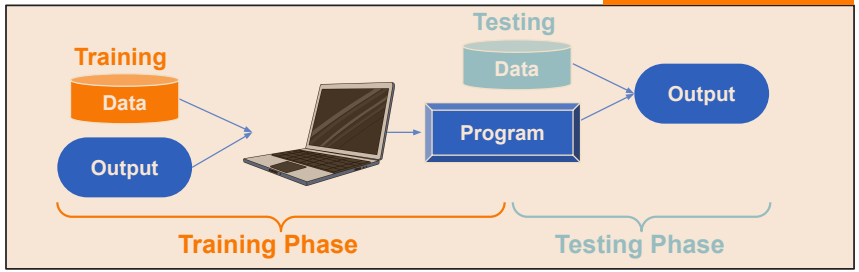
General rule:

- We want the train subset to be as large as possible
  - Better ML models with more data
- We want test subset to be large enough to be representative of the whole dataset
  - Different features appear in the same proportion

Typically,

- 80% for **training**, 20% for **testing**

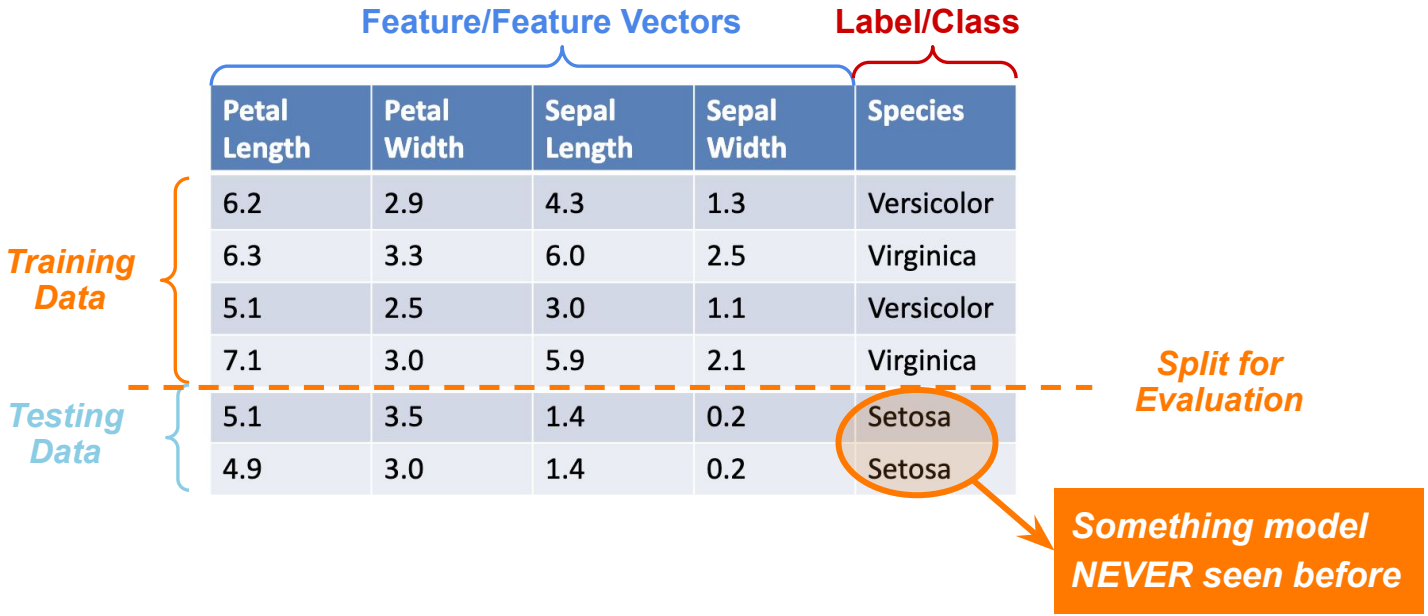
Remember?



Create a Test Set - *Sampling*

What could go wrong?

- If we just pick first 80% for **training**, remaining 20% for **testing**



Create a Test Set - *Sampling*

Random Sampling

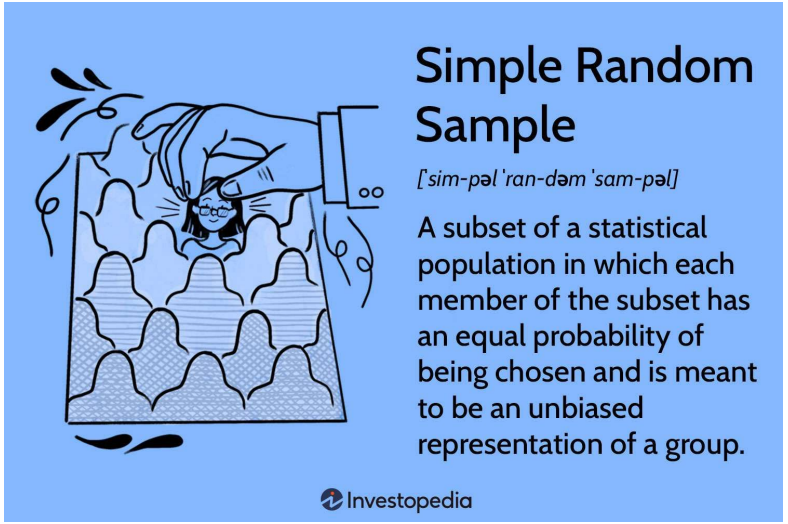
- A type of sampling method used to ensure every member of a population has an **equal chance** of being selected, reducing bias in data collection

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(myfulldata, test_size=0.2,
                                       random_state=42)
```

Problem

- Random sampling does NOT guarantee that test subset has features in the **same proportion** as the full data

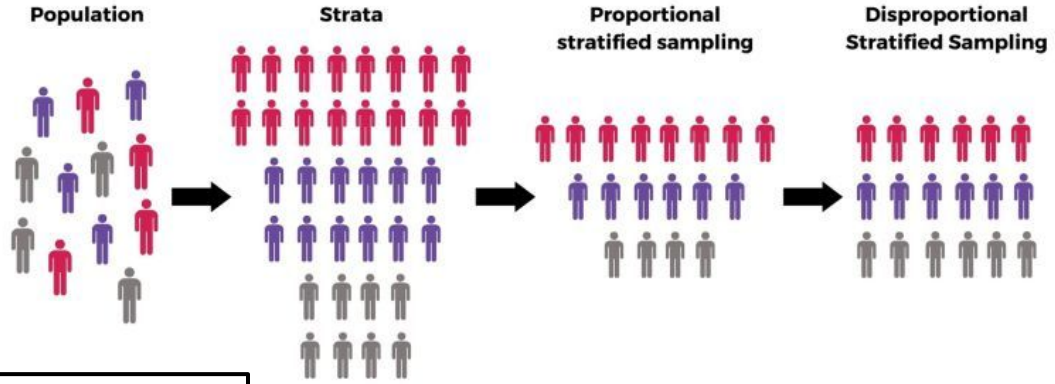


Reference:
https://www.investopedia.com/terms/s/simple-random-sample.asp

Create a Test Set - *Sampling*

Stratified Sampling

- Selecting a sample in which the population is first divided into homogeneous subgroups, (or ***strata***) based on certain characteristics that are relevant to the study
- It take in an additional column parameter and ensures test and full dataset have the same proportion of the column values




```
train_set, test_set = train_test_split(myfulldata, test_size=0.2,
                    random_state=42, stratify=myfulldata["col_name"])
```

Reference:
https://tgmresearch.com/stratified-sampling.html

Create a Test Set - *Cross Validation*

A single **train-test split** divides your data into one training set and one testing set

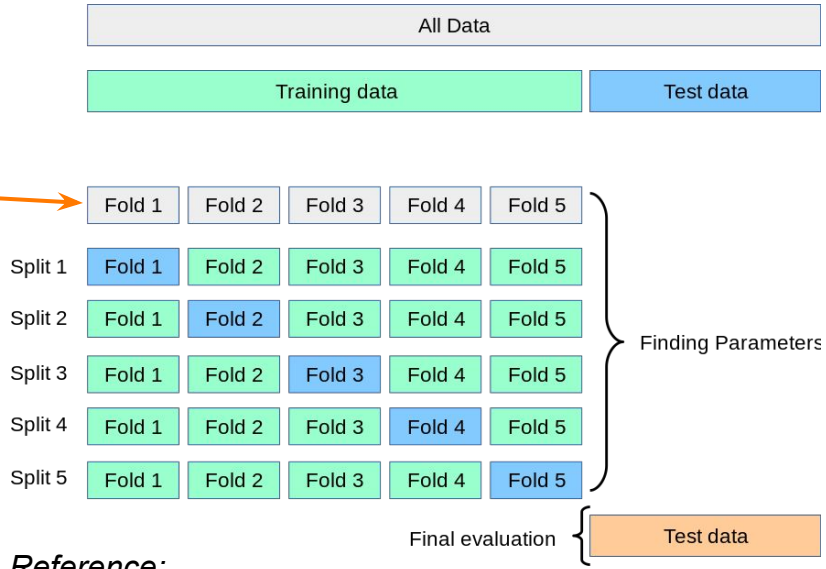
- while **cross-validation (CV)** divides your data into multiple parts (folds) to train and test the model iteratively

Common approach: ***k-fold cross-validation***

- *k*: the number of fold

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X_train, y_train, cv=5)
```



Reference:
https://scikit-learn.org/stable/modules/cross_validation.html

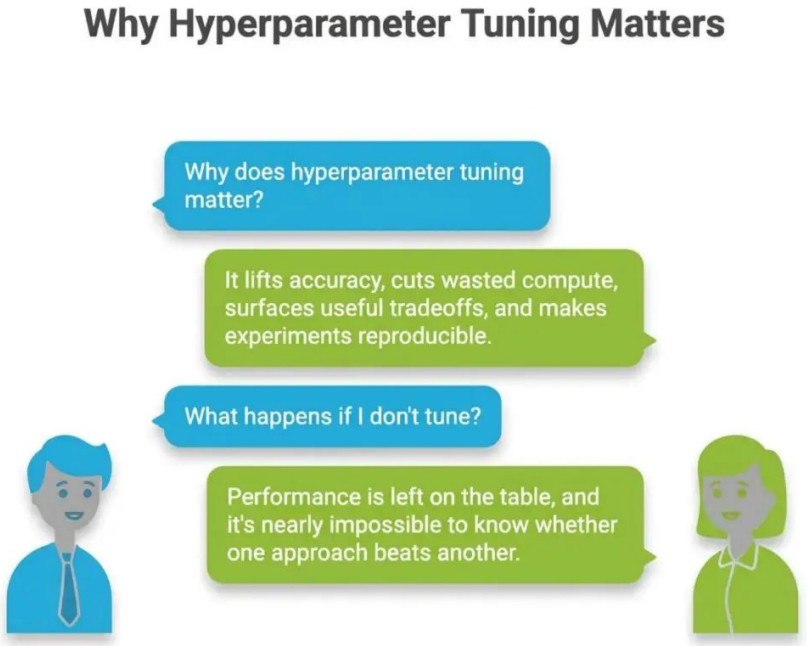
Chapter 3

Model Evaluation - Classification

- Binary Classification
- Confusion Matrices
  - Accuracy / Error
  - Confusion Matrix
  - Precision, Recall, F1 Score
  - ROC / AUC
- Multi-class Evaluation

Why Evaluation?

- Quantitatively compare different models for a given dataset
  - Which one is better?
- Identify optimal values for **hyperparameters** (settings that control the training algorithm itself)
  - Hyperparameters of the ML model
  - Hyperparameters of pre-processing steps

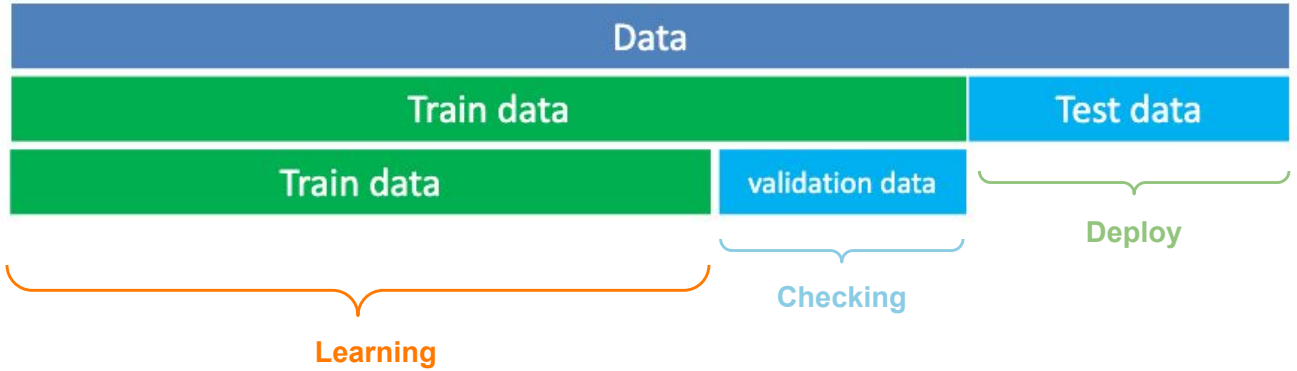


Validation Set

Split data into three

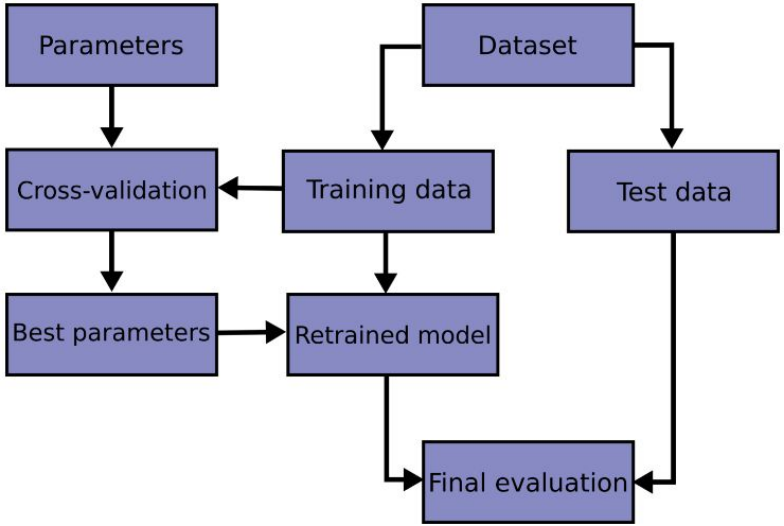
- Train data
  - Train
  - **Validation (testing the trained model)**
- Test data
  - To be used only once

Prevent exposing test data upfront



Cross validation in the full process

- After best model size is found from the error rate curve on validation data, re-train on all training data to get final model for deployment
- Evaluate final model on the test data (not used before) to estimate true generalization error (to unseen examples)



Evaluation - *How?*

Binary Classification

- Ground truth/prediction can take one of two values:
  - Generic labels: **Positive** or **Negative**

Features | Labels

Petal Length	Petal Width	Sepal Length	Sepal Width	Species
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa
6.2	2.9	4.3	1.3	Versicolor
6.3	3.3	6.0	2.5	Virginica
5.1	2.5	3.0	1.1	Versicolor
7.1	3.0	5.9	2.1	Virginica

Training data

5.1	3.5	1.4	0.2	Setosa	?
7.1	3.0	5.9	2.1	Virginica	?

Test data

Ground Truth | Predictions

*If truth and prediction **match**, this is called "True"*

- *If not, it is called "False"*

*If model predict **positive class**, this is called "Positive"*

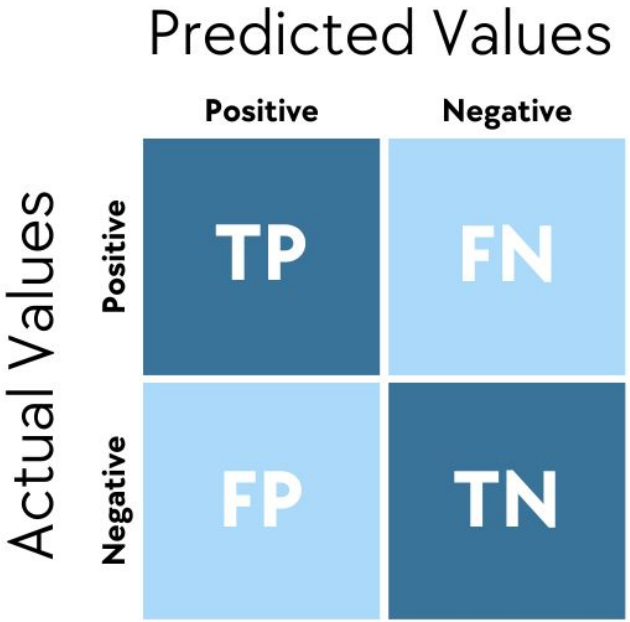
- *If the model predicted the **negative class**, then it is called "Negative"*

Classifier Evaluation Metrics - Confusion Matrix

Compare ground truth with predictions

- Four possible outcomes (for binary classification)
  - **True** Positive (i.e., TP)
  - **True** Negative (i.e., TN)
  - **False** Positives (i.e., FP)
  - **False** Negatives (i.e., FN)

- Also known as **Contingency Table**

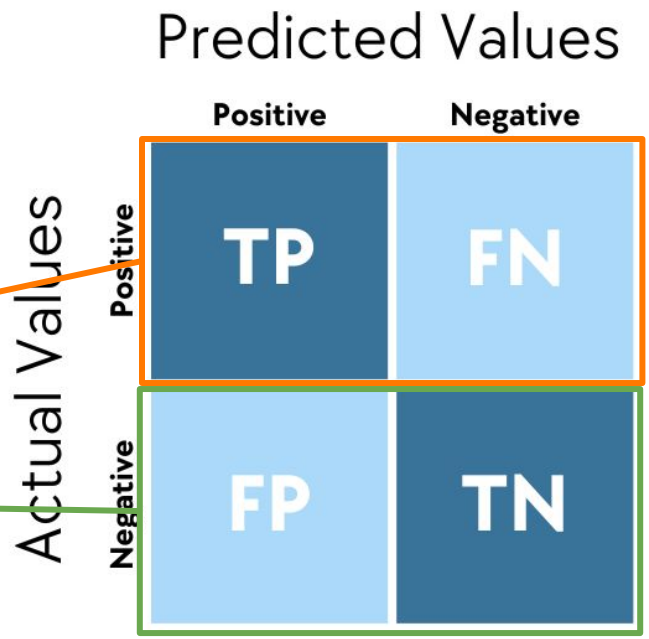



```
from sklearn.metrics import confusion_matrix

tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
```

Classifier Evaluation Metrics - *Accuracy* & *Error Rate*

- Accuracy: percentage of test set data that are correctly classified
  - Accuracy = (TP + TN) / All
- Error rate: 1 – accuracy, or
  - Error rate = (FP + FN) / All



Note:
- **P** = TP + FN
- **N** = FP + TN

All = P + N
= TP + FN + FP + TN

Classifier Evaluation Metrics - *Be careful of "Accuracy"*

Class Imbalance Problem:

- One class may be ***rare***
  - e.g., fraud, or HIV-positive

- Accuracy is dominated by the larger set (either positives/negatives) and favors trivial classifiers
  - e.g., if only 5% of items are truly positive, then a classifier that always outputs "negative" is 95% accurate

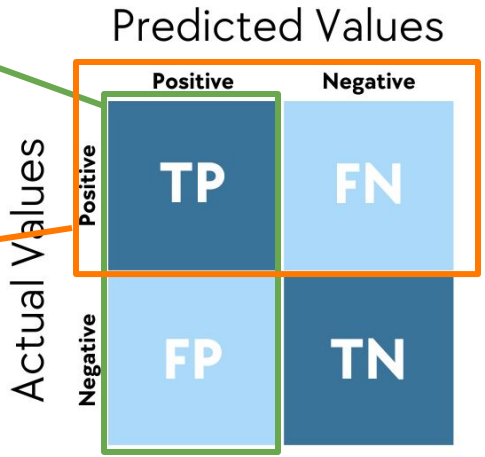
Classifier Evaluation Metrics - *Precision* & *Recall*

- Precision: *exactness*
  - what % of tuples that the classifier labeled as positive are actually positive?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Recall: *completeness*
  - what % of positive tuples did the classifier label as positive?

$$\text{Recall} = \frac{TP}{P} = \frac{TP}{TP + FN}$$



Note: *Perfect score is **1.0***

- Inverse relationship between precision & recall

Classifier Evaluation Metrics - *F measures*

Problem

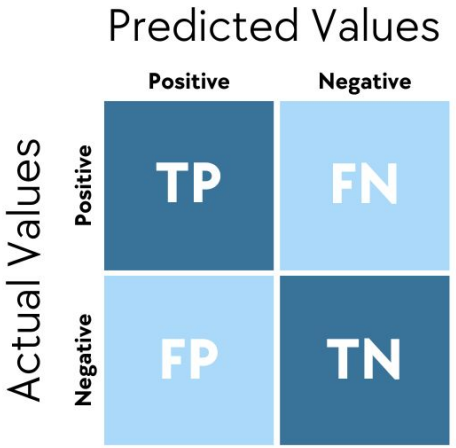
- **High Precision** but **Low Recall**
- **Low Precision** but **High Recall**

$$F = \frac{2 \times precision \times recall}{precision + recall}$$

F measures (F_1 or F-score)

- Combines Precision and Recall into one metric
  - harmonic mean of precision and recall

- Useful when both FP and FN matter
  - *High F-score*: both Precision and Recall to be high



Classifier Evaluation Metrics - *Sensitivity* & *Specificity*

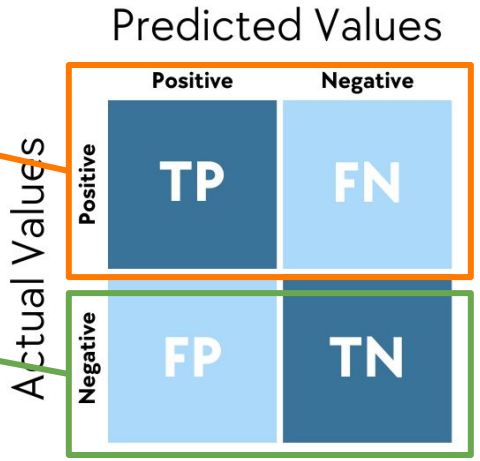
- Sensitivity: *True Positive Rate* ← **Which is your Recall :)**
  - Of all the actual positive cases, how many did the *model correctly identify*?

- Specificity: *True Negative Rate*
  - Of all the actual negative cases, how many did the *model correctly identify as negative?*

Sensitivity = TP / TP + FN
= TP / P

} **= Recall**

Specificity = TN / TN + FP
= TN / N



False Positive Rate

- Ratio of negative instances that are incorrectly classified as positive

Practice!!!

Given all the formula, calculate:

- Accuracy
- Error rate
- Precision of cancer = yes
- Recall of cancer = yes
- F_1-score of cancer = yes

Actual Class\Predicted class	cancer = yes	cancer = no
cancer = yes	**90**	**210**
cancer = no	**140**	**9560**

Actual Class\Predicted class	cancer = yes	cancer = no	Total	Recognition(%)
cancer = yes	**90**	**210**	300	30.00 (*sensitivity*)
cancer = no	**140**	**9560**	9700	98.56 (*specificity*)
Total	230	9770	10000	96.50 (*accuracy*)

Precision = 90/230 = 39.13% *Recall* = 90/300 = 30.00%

Practice!!!

What happen to multiple labels?

- “Positive” class is **High Risk**

Credit score	Income ($)	Risk
600	50,000	High
650	60,000	Low
800	55,000	Low
550	55,000	Low
660	50,000	High
750	58,000	Low

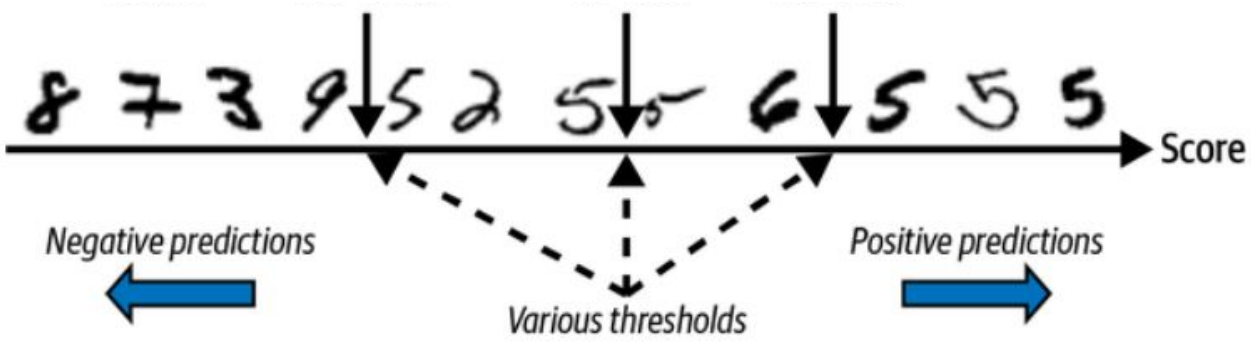
Model Prediction = [‘High’, ‘High’, ‘Low’, ‘High’, ‘High’, ‘High’]

Given all the formula, calculate:

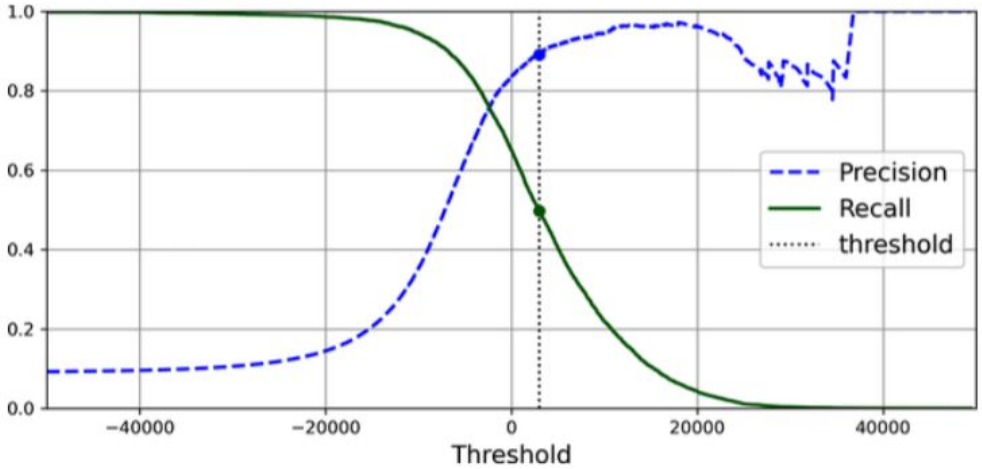
- Accuracy
- Error rate
- Precision
- Recall
- F_1-score

Precision/Recall Trade-off

- Most models compute a score based on a decision function
  - If that score is greater than a **threshold**, it assigns the instance to the positive class
  - otherwise, it assigns it to the negative class
- Changing this threshold varies the precision & recall



Precision-Recall Curve




```
y_scores = cross_val_predict(sgd_clf, X_train, y_train, cv=3, method="decision_function")

from sklearn.metrics import precision_recall_curve
precisions, recalls, thresholds = precision_recall_curve(y_train, y_scores)

plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)
plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)
```